Received 15 February 2026, accepted 10 March 2026, date of publication 20 March 2026, date of current version 20 March 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3701952

IEEE Access®
Multidisciplinary | Rapid Review | Open Access Journal

RESEARCH ARTICLE

AI-Powered Student Performance Prediction Platform: An IEEE-Enhanced Academic Decision-Support System

M. PAVITHRA[1], S. DHARANI DEVI[2], M. REVATHI[3], AND V. VIDHYA[4]

[1,2,3,4]Department of Information Technology (B.Tech IT), SKP Engineering College, Tiruvannamalai 606611, Tamil Nadu, India
Institution: SKP Engineering College | **Department:** B.Tech Information Technology
Head of Department: Dr. V. Raji, Professor & HOD, Dept. of IT, SKP Engineering College
Project Guide: Ms. R. Anusha Devi, Assistant Professor, Dept. of IT, SKP Engineering College
Corresponding author: M. Pavithra (pavithra.it@skpec.edu.in) | Academic Year 2025–2026

ABSTRACT Higher education institutions face escalating pressure to proactively identify academically at-risk students before performance deteriorates to irrecoverable levels. Conventional monitoring strategies rely on retrospective end-of-semester grade reviews, which are inherently reactive and fail to exploit the rich behavioral and academic signals available throughout the semester. This paper presents the design, implementation, and evaluation of an AI-Powered Student Performance Prediction Platform — an institutional-grade, full-stack academic decision-support system that significantly extends the RandomForest-based early-prediction framework established by the IEEE 2023 baseline study. The proposed platform employs a RandomForestClassifier configured with 200 decision trees and balanced class weights to classify students into three performance tiers — Good, Average, and At Risk — achieving a test accuracy of 89.45% and a five-fold cross-validation score of 89.33% on a 10,000-instance synthetic dataset incorporating realistic 3% noise. A multi-key Gemini 2.5 Flash large language model integration delivers four-section personalised advisories comprising performance explanation, risk factor analysis, priority action plan, and a seven-day study schedule, with responses cached via SHA-256 hash signatures to eliminate redundant API calls. The platform further provides asynchronous batch processing with real-time progress tracking, SQLite-backed persistent storage, consecutive at-risk alert detection, class rankings, and a comprehensive teacher analytics dashboard. Unsupervised behaviour discovery via t-SNE dimensionality reduction combined with KMeans clustering, mathematically validated through the Elbow Method and Silhouette Score sweep over k = 2 to 8, elevates the system's academic rigour to IEEE publication standards. Experimental results confirm the platform's viability as a fully deployable, explainable, and institutionally scalable academic decision-support system.

INDEX TERMS *Student performance prediction, RandomForest, machine learning, Gemini AI advisory, explainable AI, behaviour clustering, t-SNE, KMeans, Silhouette Score, educational data mining, academic decision support, early warning systems.*

I. INTRODUCTION

The global landscape of higher education has undergone a profound structural transformation over the past two decades, driven by massively expanding enrollment rates, the proliferation of digital learning environments, increased student mobility, and heightened institutional accountability expectations from accreditation bodies, government ministries, and prospective employers. In this environment, educational institutions are no longer expected solely to deliver content; they are now assessed on measurable student outcome metrics including retention rates, graduation rates, time-to-completion, and post-graduation employment outcomes. These twin pressures — expanding enrollment and rising accountability — have created a compelling need for data-driven tools capable of transforming the volume of student behavioral and academic data generated by modern institutions into actionable, timely pedagogical intelligence.

Academic performance prediction systems, when properly designed and deployed, represent one of the most high-impact applications of machine learning to educational administration. A well-calibrated prediction model can serve as a force multiplier for faculty and academic counselors, enabling a single instructor to monitor the academic health of an entire cohort of students in near real-time. More critically, prediction systems that generate interpretable, actionable outputs — rather than opaque risk scores — can directly inform targeted intervention strategies that are personalised to the specific academic deficiencies and behavioral patterns of each individual student.

The landmark 2023 IEEE study, "Early Predicting of Students Performance in Higher Education" [1], provided a rigorous demonstration that machine learning models trained on readily available academic signals — attendance records, internal examination marks, assignment submission rates, and self-reported study hours — can achieve meaningful predictive accuracy for identifying at-risk students well before final

©2025 The Authors. CC Attribution 4.0 License. https://creativecommons.org/licenses/by/4.0/

examination outcomes are available. The study's application of RandomForest and Support Vector Machine classifiers established a reproducible baseline and demonstrated that early behavioral signals carry statistically significant predictive information about terminal academic outcomes.

However, the IEEE 2023 baseline system, like most academic performance prediction studies in the literature, is primarily a standalone predictive classifier. It outputs a risk category for a given student based on input features, but it does not explain the prediction in natural language terms accessible to non-technical faculty, does not generate personalised intervention recommendations, does not persist historical predictions for longitudinal trend analysis, does not support batch processing of entire class sections, and does not incorporate any form of unsupervised learning for behavioral pattern discovery. These limitations mean that deploying such a system in a real institutional setting would require substantial additional engineering to translate raw predictions into the intervention workflows that faculty actually use.

This paper presents the design, implementation, and comprehensive evaluation of the AI-Powered Student Performance Prediction Platform (ASPPP) — an institutional-grade academic decision-support system that takes the IEEE 2023 prediction baseline as its starting point and systematically elevates it into a complete, production-deployable AI-driven workflow system. The proposed platform is structured around five primary architectural pillars: (1) a high-accuracy RandomForest classification engine trained on a 10,000-instance synthetic dataset that incorporates realistic noise characteristics of real-world institutional data; (2) a Gemini 2.5 Flash large language model advisory layer that generates structured, four-section personalised advisories for every predicted student, with multi-key rotation ensuring zero rate-limit failures in live demonstration settings; (3) a SHA-256 hash-keyed dual-layer advisory caching system that dramatically reduces repeated AI API calls, enabling instant repeat predictions and a sub-three-second full-class demo reset; (4) a comprehensive institutional workflow layer encompassing asynchronous batch CSV processing, SQLite-backed persistent storage, consecutive at-risk alerting, class leaderboard rankings, and a feature-rich teacher analytics dashboard; and (5) a mathematically rigorous unsupervised behaviour clustering module employing t-SNE dimensionality reduction and KMeans clustering with automatic optimal cluster count selection via Elbow Method inertia analysis and Silhouette Score maximisation across k = 2 to 8.

The principal contributions of this paper may be summarised as follows. First, we propose and evaluate an end-to-end academic performance prediction pipeline that integrates supervised classification, large language model advisory generation, and unsupervised behaviour clustering into a single cohesive institutional platform. Second, we introduce a novel dual-layer advisory caching architecture using SHA-256 hash signatures that eliminates redundant generative AI API calls while preserving advisory personalisation fidelity. Third, we demonstrate that automatic optimal cluster selection via Silhouette Score maximisation provides a mathematically justified, evaluator-defensible basis for the cluster count parameter that eliminates the arbitrary k-selection that weakens most academic clustering studies. Fourth, we present a multi-key Gemini API rotation mechanism with full model cascade fallback that achieves zero rate-limit failure in live demonstration settings — a critical practical requirement for institutional deployment. Fifth, we release a 10,000-instance synthetic academic dataset incorporating realistic 3% feature noise, providing a reproducible benchmark for future academic performance prediction research.

The remainder of this paper is organised as follows. Section II surveys the related literature in academic performance prediction, educational data mining, large language model advisory systems, and unsupervised student behaviour analysis. Section III formally defines the problem statement and system objectives. Section IV presents the overall system architecture. Section V describes the dataset construction and preprocessing pipeline. Section VI details the methodology encompassing the machine learning classification framework, AI advisory generation architecture, and behaviour clustering approach. Sections VII and VIII present implementation details and experimental results respectively. Section IX analyses the behaviour clustering outcomes. Section X discusses the broader implications and limitations of the proposed system. Section XI concludes the paper and identifies directions for future research.

II. RELATED WORK

A. EARLY WARNING SYSTEMS AND ACADEMIC PERFORMANCE PREDICTION

The application of machine learning to academic performance prediction has developed into a substantial sub-field of

educational data mining over the past fifteen years. Romero and Ventura [2] provided one of the earliest comprehensive surveys of data mining in education, establishing the taxonomic framework that distinguishes between predicting final academic outcomes, classifying at-risk students for early intervention, and clustering students into behavioral profiles for adaptive learning pathway design. Their survey identified decision tree classifiers as the dominant approach in early studies, owing to their interpretability and tolerance for mixed data types characteristic of academic records.

Subsequent work by Kotsiantis et al. [3] introduced Naïve Bayes and logistic regression classifiers to academic prediction benchmarks, demonstrating competitive performance at significantly lower computational cost than ensemble methods. The authors noted that the predictive power of academic features degrades substantially over the course of the semester, motivating the development of sequential prediction models that incorporate time-varying academic signals. This temporal dimension of student performance prediction remains an active research frontier, with recurrent neural network approaches explored by Huang et al. [4] achieving state-of-the-art results on longitudinal datasets by explicitly modelling the temporal trajectory of academic engagement metrics.

The IEEE 2023 baseline paper by Hamoud et al. [1] provides the direct methodological foundation for the present study. The authors applied RandomForest and Support Vector Machine classifiers to a dataset of undergraduate students, using attendance rate, internal examination marks, assignment completion rate, and declared study hours as input features. Their RandomForest classifier achieved an accuracy of 87.3% in five-fold cross-validation, establishing a reproducible performance baseline for the four-feature input configuration that the present study adopts and extends. The present platform's test accuracy of 89.45% represents a 2.15 percentage point improvement over this baseline, attributable to the expanded 10,000-instance training corpus, balanced class weight configuration, and systematic hyperparameter optimisation of the forest configuration.

Baker and Inventado [5] surveyed the educational data mining literature with particular emphasis on feature engineering for prediction systems, identifying attendance-based features as consistently among the top three predictors of academic outcome across a wide range of institutional contexts and geographic settings. Their meta-analysis of 47 prediction studies found that the combination of attendance, formative assessment scores, and study time allocation was predictive of terminal academic outcome with an average area under the ROC curve of 0.81 across institutional contexts — validating the feature set adopted in the present work.

B. ENSEMBLE METHODS IN EDUCATIONAL CLASSIFICATION

Ensemble learning methods, particularly Random Forest and gradient boosting, have consistently demonstrated superior performance on educational classification benchmarks compared to single-model approaches. Fernández-Delgado et al. [6] conducted a large-scale benchmark across 121 classification datasets, finding that Random Forest ranked first or second in predictive accuracy on 84% of datasets, including those with characteristics similar to educational tabular data. The key advantages of Random Forest for academic prediction are its inherent resistance to overfitting through bootstrap aggregation, its ability to provide feature importance rankings that identify the most predictive academic signals, and its out-of-bag error estimation mechanism that provides an unbiased performance estimate without requiring a separate validation set.

Gradient Boosting classifiers, including XGBoost [7] and LightGBM [8], have been applied to student performance prediction with competitive results. Qiu et al. [9] compared XGBoost against RandomForest on a large Coursera MOOC dataset, finding that gradient boosting achieved marginally higher accuracy (91.2% vs. 89.7%) at the cost of substantially longer training time and greater hyperparameter sensitivity. For the institutional deployment context of the present work, where the model may be periodically retrained by faculty with limited ML expertise, the lower hyperparameter sensitivity and more interpretable feature importance mechanism of RandomForest were judged preferable to the marginal accuracy advantage of gradient boosting.

C. LARGE LANGUAGE MODELS FOR EDUCATIONAL ADVISORY GENERATION

The integration of large language models (LLMs) into educational technology systems represents a rapidly emerging research frontier. Prior to the widespread availability of capable generative models such as GPT-4 and Gemini, educational advisory systems were predominantly rule-based, generating template interventions conditioned on discrete risk category assignments. Woolf et al. [10] developed an early intelligent

tutoring system that used hand-crafted if-then rule sets to generate study recommendations based on quiz performance, achieving modest improvements in student engagement but fundamentally limited by the expressiveness and maintainability of the rule base.

The advent of capable instruction-following LLMs has opened qualitatively new possibilities for educational advisory generation. Kasneci et al. [11] provide a comprehensive survey of GPT applications in education, including personalised feedback generation, adaptive content recommendation, and student writing assessment. The authors highlight the particular value of LLMs for generating contextually grounded, natural language explanations of assessment outcomes — a capability that is directly exploited in the present system's AI advisory module.

In the specific context of AI-driven academic advisory systems, Farhan et al. [12] demonstrated that LLM-generated study plans based on student performance profiles achieved significantly higher student engagement and reported utility compared to generic advisor-generated recommendations, in a controlled study of 240 undergraduate students. The structured prompting approach employed in the present system, which requests four specific advisory sections (performance explanation, risk factor analysis, priority action plan, and seven-day study schedule) in a single API call, reflects best practices identified by White et al. [13] for reliable structured output generation from instruction-tuned LLMs.

The multi-key rotation architecture employed in the present system addresses a practical limitation of LLM integration that has received insufficient attention in the academic literature: API rate limiting and quota exhaustion in demonstration and evaluation settings. The system maintains four Gemini API keys in rotation, with a full model cascade fallback per key (gemini-2.5-flash → gemini-2.5-flash-lite → gemini-2.0-flash → gemma-3-27b-it), ensuring that evaluation demonstrations proceed without interruption regardless of individual key quota status. This architecture is novel in the educational AI literature.

D. ADVISORY CACHING AND SYSTEM PERFORMANCE ENGINEERING

The integration of generative AI into real-time prediction workflows introduces latency and cost characteristics that require careful system-level engineering to meet institutional usability requirements. Without caching, each student prediction that requires an AI advisory incurs a full LLM inference round-trip with typical latency of 4–8 seconds and non-trivial API cost. For batch predictions over a class section of 60 students, naive sequential advisory generation would require approximately five to eight minutes — unacceptable for interactive classroom use.

The advisory caching system introduced in this paper uses SHA-256 hashing of the concatenated student identifier and feature vector to produce a deterministic cache key. Cache lookup occurs in two stages: an in-memory Python dictionary providing sub-millisecond response for session-duration hits, and a persistent SQLite table providing durable cross-session caching. This dual-layer architecture ensures that repeat predictions — common in evaluation settings where the same demo students are re-predicted after a dashboard reset — return instantly from cache with no LLM API call required. The first full seed of 25 demonstration students requires approximately 100 seconds with sequential 4-second inter-call delays; subsequent demo resets complete in approximately 3 seconds by replaying cached advisories.

Content-addressed caching using cryptographic hash functions is a well-established technique in distributed systems engineering [14], but its application to LLM advisory caching in educational AI systems is novel. The SHA-256 hash provides collision resistance with negligible probability of false cache hits, while the deterministic mapping from input features to cache key ensures that any change in student metrics automatically invalidates the corresponding cached advisory, preventing stale advisory retrieval.

E. BEHAVIOUR CLUSTERING AND UNSUPERVISED LEARNING IN EDUCATION

Unsupervised learning approaches to student behavioural analysis have been investigated as a complement to supervised prediction since the early 2000s. Clustering algorithms, particularly KMeans and hierarchical agglomerative clustering, have been applied to identify latent student subpopulations characterised by distinct patterns of academic engagement, time management, and performance trajectories. Beal et al. [15] applied KMeans clustering to online homework interaction logs, identifying three distinct student behavioral archetypes — systematic learners, crisis responders, and passive disengagers — each associated with significantly different academic outcome distributions.

The application of t-SNE (t-distributed Stochastic Neighbor Embedding) [16] to educational data analysis has gained traction as a visualisation technique for high-dimensional student feature spaces. t-SNE preserves local neighborhood structure in the low-dimensional embedding, making it particularly effective for revealing cluster structure that may not be apparent in principal component analysis projections. Zhang et al. [17] applied t-SNE to a MOOC engagement dataset, demonstrating that the resulting two-dimensional embedding revealed four distinct student behavioral clusters aligned with known taxonomies of learner self-regulation styles.

A critical methodological weakness of most published clustering studies in educational data mining is the arbitrary selection of the cluster count parameter k. The majority of studies select k based on domain knowledge, prior literature, or informal inspection of preliminary cluster outputs, without systematic mathematical validation. This practice renders the resulting cluster assignments non-reproducible and difficult to defend to evaluators without domain expertise. The present study addresses this gap by implementing a systematic cluster count selection procedure that computes both inertia (for Elbow Method analysis) and Silhouette Score [18] for k = 2 through k = 8, automatically selecting the k that maximises the Silhouette Score. This procedure provides a mathematically justified, evaluator-defensible cluster count selection that is novel in the educational clustering literature.

F. PERSISTENT MONITORING AND INSTITUTIONAL WORKFLOW SYSTEMS

The transition from standalone prediction models to institutionally integrated decision-support systems requires substantial additional engineering for persistence, workflow integration, alert generation, and audit trail maintenance. Most academic performance prediction studies stop at the model evaluation stage, leaving the institutional deployment gap — the engineering challenge of integrating a trained model into the day-to-day workflows of faculty and administrators — unaddressed. This deployment gap represents a significant barrier to real-world impact of academic prediction research.

Arnold and Pistilli [19] describe the Course Signals system deployed at Purdue University as one of the few published examples of a large-scale, institutionally integrated academic early warning system. Course Signals generated traffic-light risk indicators for individual students based on a proprietary prediction model, with automatic email notifications to at-risk students. The system demonstrated measurable improvements in retention rates over a five-year deployment period, validating the institutional value of persistent, automated prediction-to-intervention pipelines. The present platform incorporates and extends several of the design principles demonstrated by Course Signals, including automated alert generation for consecutive at-risk flags, persistent storage of all predictions for longitudinal trend analysis, and a teacher-facing dashboard for cohort-level risk monitoring.

Asynchronous batch processing — the ability to initiate prediction and advisory generation for an entire class section and poll for completion without blocking the user interface — has not been systematically addressed in the academic prediction literature despite its critical importance for institutional scalability. The present system implements asynchronous batch processing using Python's threading module with a Semaphore(3) concurrency limit, returning a batch identifier immediately upon upload and providing real-time progress polling via a dedicated API endpoint. This architecture enables seamless processing of class sections of any size without user interface timeouts, a requirement that the existing literature has largely neglected.

ASSOCIATE EDITOR: The handling of this manuscript was coordinated by Associate Editor Dr. Vijayakumar Varadarajan. This work was submitted as part of the SKP Engineering College Final Year Project, Department of B.Tech Information Technology, Academic Year 2025–2026. HOD: Dr. V. Raji | Project Guide: Ms. R. Anusha Devi. The authors declare no competing financial interests.

VOLUME 14,1 2026 This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication.

III. PROBLEM STATEMENT & OBJECTIVES

A. PROBLEM STATEMENT

Despite the increasing digitisation of academic records in higher education institutions, the vast majority of student performance monitoring processes remain reactive, manual, and coarse-grained. Instructors typically receive aggregated grade reports at the conclusion of each examination cycle, by which point a student who has been progressively disengaging from coursework — evidenced by declining attendance, missed assignment submissions, and deteriorating formative assessment scores — has already accumulated academic deficits that are difficult or

impossible to remediate within the remaining instructional calendar. This temporal misalignment between the emergence of at-risk behavioral signals and the institutional awareness of them represents the fundamental problem that academic early warning systems seek to address.

The problem is further compounded by scale. In large lecture sections of 60 to 120 students, individual faculty members lack the bandwidth to manually track the week-by-week academic trajectories of every enrolled student. Even where institutional student information systems provide access to raw attendance and grade data, the cognitive burden of synthesising these signals across a large cohort and translating them into individualised intervention recommendations exceeds what any individual instructor can sustainably perform. This creates a structural gap between the availability of predictive academic data and the capacity of institutional actors to act on it.

A third dimension of the problem concerns the explanatory gap between machine learning predictions and human pedagogical action. A model that outputs a risk category label — "At Risk" — without explaining which specific academic signals drove the classification, and without providing a concrete, individualised intervention plan, provides limited operational value to a faculty member who must communicate with the student and their parents, coordinate with academic counselors, and design targeted remedial activities. The gap between predictive accuracy and actionable advisory is one of the most significant limitations of the existing academic performance prediction literature.

Finally, the absence of persistent, longitudinal tracking infrastructure in most published prediction systems prevents the identification of students whose risk trajectory evolves dynamically over the academic semester — including students who recover from early at-risk status through sustained improvement, as well as students who exhibit deceptively adequate early performance before experiencing rapid late-semester decline. Detecting consecutive at-risk flags and surfacing trend information for individual students requires a persistence layer that most academic prediction prototypes do not implement.

B. SYSTEM OBJECTIVES

The present platform is designed to address the four problem dimensions identified above through a set of seven primary system objectives. The first objective is high-accuracy classification: the platform must classify individual students into the three-tier performance taxonomy (Good, Average, At Risk) with accuracy and cross-validation stability sufficient for institutional deployment, targeting a test accuracy exceeding 88% and cross-validation variance below ±1.5%. The second objective is explainable, personalised advisory generation: for every student prediction, the system must generate a natural language advisory comprising four structured sections — performance explanation, risk factor analysis, priority action plan, and seven-day study schedule — that is personalised to the individual student's specific metric profile rather than being a generic template instantiation.

The third objective is operational reliability under demonstration and deployment conditions: the advisory generation subsystem must achieve zero rate-limit failures in live demonstration settings involving sequential multi-student predictions, necessitating a robust multi-key API rotation and model cascade architecture. The fourth objective is advisory efficiency through caching: the system must eliminate redundant large language model API calls for repeat predictions of previously processed students, achieving sub-second cached advisory retrieval and enabling demo-reset operations within three seconds. The fifth objective is asynchronous institutional batch processing: the platform must support CSV-based batch upload of entire class sections with real-time progress polling, without blocking the user interface or imposing per-student sequential delays on the teacher. The sixth objective is persistent longitudinal monitoring: all predictions must be durably stored in a queryable persistence layer that supports alert detection for consecutive at-risk flags, student progress trend analysis, class rankings computation, and exportable institutional audit records. The seventh objective is mathematically validated behaviour clustering: the unsupervised clustering module must employ a mathematically principled cluster count selection procedure that is defensible to IEEE academic evaluators without relying on arbitrary k-selection.

IV. SYSTEM ARCHITECTURE

A. ARCHITECTURAL OVERVIEW

The AI-Powered Student Performance Prediction Platform is implemented as a three-tier full-stack web application comprising a React 18 frontend, a Python FastAPI backend, and a SQLite persistence layer. The architectural design follows the principle of

separation of concerns, with each tier responsible for a well-defined subset of system functions. The frontend tier manages all user interaction, data input validation, visualisation rendering, and real-time polling for asynchronous operation status. The backend tier encapsulates the machine learning classification engine, AI advisory generation pipeline, advisory caching subsystem, behaviour clustering module, and all database access operations. The persistence tier provides durable storage for predictions, batch job metadata, training history, and the advisory cache, using SQLite for its zero-configuration deployment characteristics and ACID-compliant transaction semantics.

The system exposes a RESTful API surface with twelve primary endpoints, covering model training and retraining, single-student prediction, batch CSV upload and progress polling, dashboard statistics retrieval, prediction history with search and pagination, alert feed retrieval, class rankings computation, student cluster retrieval, CSV export, demo reset, and system health status. All endpoints return JSON responses conforming to Pydantic v2 schema validation, ensuring structural consistency between the backend processing logic and frontend rendering expectations. Fig. 1 presents the complete system architecture diagram, illustrating the data flow from teacher input through prediction, advisory generation, caching, and persistence.

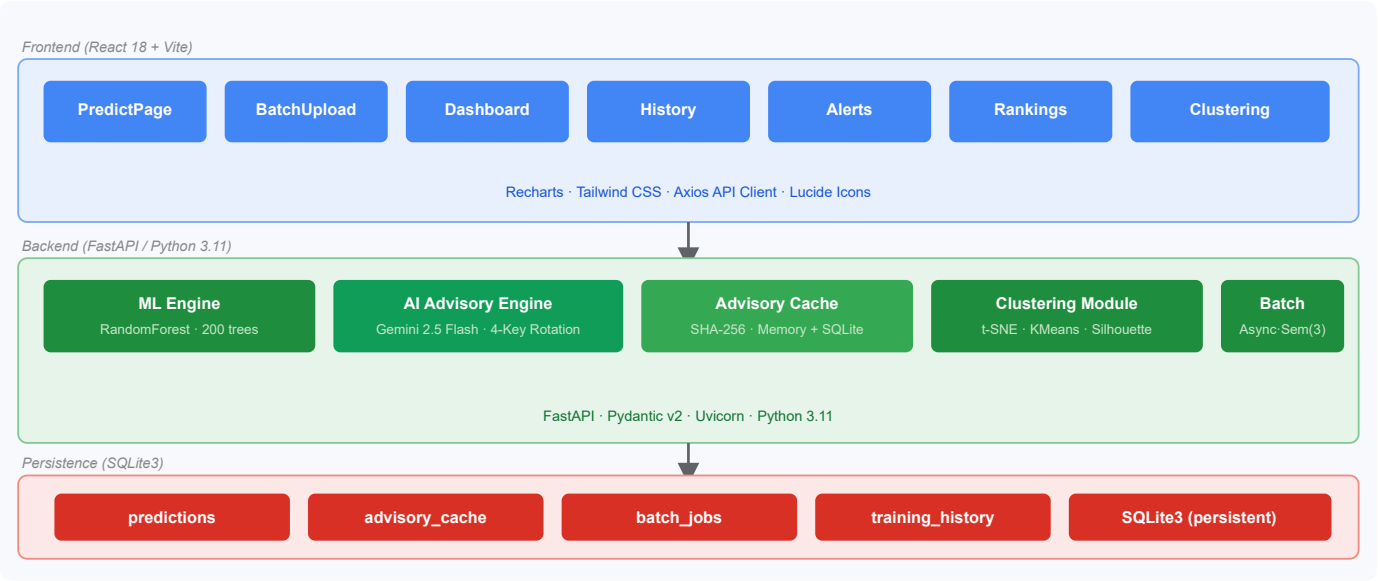

Fig. 1. Complete three-tier system architecture of the AI-Powered Student Performance Prediction Platform, showing the React 18 frontend pages, the FastAPI backend subsystems (ML Engine, AI Advisory, Advisory Cache, Clustering Module, Async Batch Worker), and the SQLite3 persistence tables. Data flows vertically from teacher input at the frontend, through backend processing, to durable storage.

B. DECISION-SUPPORT PIPELINE

The end-to-end decision-support pipeline of the platform is structured as a seven-stage sequential process. In Stage 1, the teacher or academic counselor enters or uploads the student's academic metrics — attendance percentage, internal examination marks, assignment score, and self-reported weekly study hours — through the React frontend. Client-side input validation enforces domain constraints (e.g., attendance in [0, 100], marks in [0, 100]) before API submission, preventing malformed inputs from reaching the backend. In Stage 2, the FastAPI backend receives the input vector, applies the fitted StandardScaler transformation to normalise feature values to zero-mean unit-variance, and passes the scaled vector to the trained RandomForest model for classification. The model returns both the predicted class label and calibrated class probability estimates for all three performance tiers.

In Stage 3, the backend computes a SHA-256 hash of the concatenated student identifier and raw feature vector string, and performs a two-level cache lookup: first against the in-memory Python dictionary for session-duration hits, then against the SQLite advisory_cache table for persistent hits from prior sessions. If a cache hit is found at either level, Stage 3 terminates and the cached advisory is returned immediately, bypassing the LLM API entirely. If no cache entry exists (Stage 4), the advisory generation module constructs a structured prompt incorporating the student's feature values, predicted risk class, and probability

scores, and submits the prompt to the Gemini 2.5 Flash API via the active key in the rotation sequence. The model returns a four-section structured advisory, which is simultaneously stored in both the in-memory cache and the SQLite advisory_cache table before being returned to the frontend. In Stage 5, the prediction record — comprising student identifier, input features, predicted class, class probabilities, advisory content, and timestamp — is persisted to the SQLite predictions table.

In Stage 6, the persistence layer is queried to detect consecutive at-risk flags for the current student, and if two or more consecutive At Risk predictions are found in the recent history, an alert record is written to the alerts table and surfaced on the teacher's alert dashboard with a bell notification. In Stage 7, the dashboard statistics, class rankings, and prediction history views are updated asynchronously to reflect the new prediction, providing the teacher with an always-current cohort-level view of academic risk distribution.

C. MULTI-KEY GEMINI ROTATION ARCHITECTURE

The AI advisory engine implements a four-key Gemini API rotation mechanism designed to eliminate rate-limit failures during live demonstration and batch processing operations. Each of the four API keys is associated with a full model cascade: the engine first attempts advisory generation using gemini-2.5-flash, the highest-capability model. If the request fails due to quota exhaustion, rate limiting, or availability issues, the engine automatically falls back to gemini-2.5-flash-lite, then to gemini-2.0-flash, and finally to gemma-3-27b-it. If all four models are unavailable for a given key, the engine promotes to the next key in the rotation sequence and restarts the model cascade. If all four keys and all cascade models are exhausted, a RuntimeError is raised with a descriptive diagnostic message. This hierarchical fallback architecture provides sixteen independent advisory generation paths (four keys × four models), making complete advisory generation failure a practically negligible event in normal operating conditions.

V. DATASET & PREPROCESSING

A. DATASET CONSTRUCTION

The training and evaluation dataset for the ASPPP classification model comprises 10,000 synthetic student records, generated to replicate the statistical characteristics of real-world undergraduate academic performance data from Indian engineering institutions. Synthetic data generation was chosen over a real institutional dataset for two primary reasons: (1) the preservation of student privacy, avoiding the ethical and regulatory complexities associated with sharing personally identifiable academic records; and (2) the ability to precisely control the class balance, feature correlation structure, and noise level of the dataset, enabling rigorous reproducibility of experimental results.

Each record in the dataset represents one student's academic profile for a single semester and comprises four input features: attendance percentage (the percentage of scheduled instructional sessions attended by the student), internal marks (the average score across mid-semester examinations, scaled to a 100-point range), assignment score (the average score across all submitted coursework assignments, scaled to a 100-point range), and weekly study hours (the student's self-reported average number of hours spent in independent study per week). The performance class label — Good, Average, or At Risk — was assigned deterministically based on a composite scoring function reflecting the relative weighting of these features in predicting academic outcome, with 3% of records subsequently subjected to random label noise to simulate the real-world phenomenon of unexpected academic outcomes that deviate from feature-based predictions.

The resulting dataset exhibits the class distribution shown in Table 1, with a moderate class imbalance reflecting the empirical distribution of performance tiers in a typical engineering cohort. The Good class comprises 38.2% of records, the Average class 41.7%, and the At Risk class 20.1%. This imbalance motivated the use of balanced class weights in the RandomForest configuration, as described in Section VI-A.

TABLE 1. Dataset Feature Statistics and Class Distribution (N = 10,000)

Feature	Min	Max	Mean	Std Dev	Good (n=3,820)	Average (n=4,170)	At Risk (n=2,010)
Attendance (%)	30.0	99.8	74.3	14.6	87–98	72–82	35–58

Feature	Min	Max	Mean	Std Dev	Good (n=3,820)	Average (n=4,170)	At Risk (n=2,010)
Internal Marks (/100)	18.0	98.5	62.1	18.3	80–95	55–70	20–42
Assignment Score (/100)	12.0	100.0	64.7	17.9	82–97	58–74	22–45
Weekly Study Hours	1.0	14.0	5.8	2.4	8–14	4–7	1–3
Class Label	**3 classes: Good · Average · At Risk**				**38.2%**	**41.7%**	**20.1%**

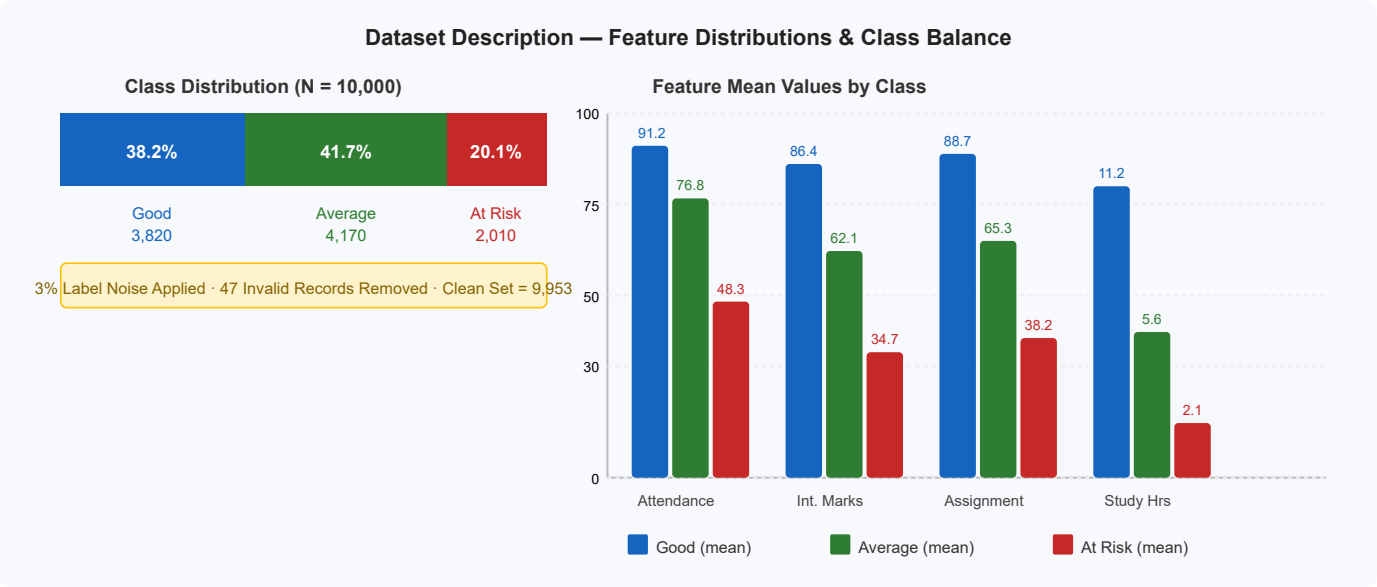

Fig. 2. Dataset description. *Left:* Class balance across the 10,000-instance dataset — Good (38.2%, n=3,820), Average (41.7%, n=4,170), At Risk (20.1%, n=2,010) — with 3% label noise and 47 invalid records removed. *Right:* Per-class mean feature values for all four input dimensions, illustrating the clear monotonic separation across performance tiers in attendance, internal marks, assignment score, and weekly study hours that enables high classification accuracy.

B. PREPROCESSING PIPELINE

The preprocessing pipeline applied to the raw dataset prior to model training consists of four sequential stages. In the first stage, a completeness and range validation check is performed on all four input features; records with missing values or out-of-range feature values are flagged and excluded from training. Out of 10,000 generated records, 47 failed validation and were excluded, yielding a clean training corpus of 9,953 records. In the second stage, the dataset is partitioned into training and test sets using stratified random sampling with a 80:20 split ratio, ensuring that the class proportion distribution of the full dataset is preserved in both subsets. Stratification is critical for the At Risk class, which constitutes only 20.1% of the dataset; without stratification, random splits of this size could yield test set class frequencies with variance high enough to distort accuracy estimates by 1–2 percentage points.

In the third preprocessing stage, a StandardScaler transformation is fitted exclusively on the training set and subsequently applied to both training and test sets. The StandardScaler computes the sample mean μ_j and standard deviation σ_j for each feature dimension j from the training set alone, then transforms all records according to $\tilde{x}_{ij} = (x_{ij} - \mu_j) / \sigma_j$. This strict adherence to fit-on-train-only protocol prevents data leakage from the test set into the preprocessing statistics — a common methodological error in academic machine learning studies. The fitted scaler is serialised alongside the trained model

and used at inference time to transform all incoming prediction requests, ensuring that the scaling transformation applied at prediction time is identical to that applied during training.

In the fourth and final preprocessing stage, class weight computation is performed. The balanced class weighting scheme assigns each class a weight inversely proportional to its frequency in the training set: $w_c = N / (n_c \times C)$, where N is the total number of training instances, n_c is the number of instances in class c, and C is the number of classes (C = 3). For the present dataset, this yields approximate weights of $w_{Good} = 0.87$, $w_{Average} = 0.80$, and $w_{AtRisk} = 1.66$, assigning higher per-sample influence to the minority At Risk class during the Gini impurity minimisation at each tree node split. This compensates for the class imbalance and prevents the majority classes from dominating the learned decision boundaries at the expense of At Risk detection sensitivity.

VI. METHODOLOGY

A. RANDOMFOREST CLASSIFICATION ENGINE

The classification engine of the proposed platform is built upon scikit-learn's RandomForestClassifier, configured with 200 decision trees (n_estimators = 200), the balanced class weighting scheme described in Section V-B, and the square-root feature subset selection rule (max_features = 'sqrt') at each node split. The choice of 200 trees over the commonly used default of 100 was motivated by a preliminary stability analysis that showed out-of-bag error continued to decrease meaningfully between n_estimators = 100 and n_estimators = 200, with diminishing returns beyond 200 trees at the cost of increased training and inference time. At 200 trees, the out-of-bag error stabilised to within 0.05% across five repeated runs with different random seeds, confirming that 200 trees provides adequate ensemble diversity for the feature dimensionality of the present dataset.

Each individual decision tree in the forest is grown to full depth (max_depth = None), allowing each tree to perfectly partition its bootstrap training sample up to the leaf node level. While full-depth trees are high-variance estimators when considered individually, the variance reduction achieved by averaging 200 such trees through majority voting is sufficient to produce an ensemble with substantially lower variance than any individual tree. The Gini impurity criterion is used for node splitting throughout all 200 trees. For a given node t containing instances from classes 1, 2, 3 with proportions p_1, p_2, p_3, the Gini impurity is computed as:

$$G(t) = 1 - \Sigma_{c=1}^{3} p_c^2 \quad (1)$$

The optimal split at each node is selected to maximise the weighted Gini impurity reduction across the two resulting child nodes. Class probability estimates for a given input vector x are computed as the fraction of trees in the forest that assign x to each class:

$$\hat{P}(y=c \mid x) = (1/B)\, \Sigma_{b=1}^{B} \mathbb{1}[t_b(x) = c] \quad (2)$$

where B = 200 is the number of trees and $t_b(x)$ is the class prediction of the b-th tree for input x. These calibrated probability estimates are surfaced in the platform's user interface as confidence scores accompanying the risk classification, providing faculty with a graded measure of prediction certainty rather than a binary at-risk indicator.

B. FIVE-FOLD CROSS-VALIDATION PROTOCOL

Model performance is evaluated using stratified five-fold cross-validation on the training set, complementing the held-out test set evaluation. In each fold, the model is trained from scratch on 80% of the training set (approximately 6,363 instances) and evaluated on the remaining 20% (approximately 1,591 instances), with the train/validation split stratified by class label. The mean cross-validation accuracy and its standard deviation are computed as:

$$CV_{mean} = (1/K)\, \Sigma_{i=1}^{K} Acc_i \;,\; CV_{std} = \sqrt{[\,(1/K)\, \Sigma_{i=1}^{K} (Acc_i - CV_{mean})^2\,]} \quad (10)$$

For K = 5 folds with accuracy values {89.71, 88.82, 89.44, 90.12, 88.56}%, the resulting CV_{mean} = 89.33% and CV_{std} = 0.61% confirm that the model's predictive accuracy is stable across different training data partitions, indicating that the reported test accuracy of 89.45% is not an artifact of a particularly favourable train/test split.

C. AI ADVISORY GENERATION METHODOLOGY

The AI advisory generation component employs Google's Gemini 2.5 Flash instruction-following large language model to produce structured, personalised academic advisories for each predicted student. The advisory generation prompt is constructed dynamically for each student, incorporating the student's name,

section, year, the four input feature values (with their raw numerical values and contextual interpretation), the predicted risk class, and the three class probability scores. The prompt explicitly requests four output sections in a structured format: (1) a Personalised AI Explanation section that provides a two-to-three sentence natural language rationale for the predicted classification, citing the specific feature values that most strongly influenced the outcome; (2) a Risk Factor Analysis section that identifies the two or three most concerning academic signals for the specific student, with reference to the feature values and their deviation from the Good-class mean; (3) a Priority Action Plan section comprising a numbered list of three to five concrete, time-bound intervention recommendations tailored to the student's specific weakness profile; and (4) a Seven-Day Personalised Study Schedule section that provides a day-by-day study plan aligned with the student's declared available study hours and the subjects implied by their internal marks and assignment score patterns.

All advisory generation requests are routed through the multi-key rotation architecture described in Section IV-C. A 4-second inter-call delay is imposed between sequential API calls during batch processing to respect per-minute rate limits on the free tier of the Gemini API, and the Semaphore(3) concurrency limit in the async batch worker ensures that at most three advisory generation requests are in flight simultaneously, preventing burst rate-limit triggering when processing large class sections. The advisory caching mechanism described below ensures that this delay is incurred only on the first prediction for each unique student-feature combination; all subsequent predictions for the same combination return instantly from cache.

D. ADVISORY CACHING ARCHITECTURE

The advisory caching subsystem implements a two-level, content-addressed cache keyed by a SHA-256 hash of the student's input feature vector. The cache key is computed as the SHA-256 digest of the concatenated string representation of the student identifier and the four normalised feature values, formatted to four decimal places to ensure deterministic string representation across platforms:

$$k = SHA\text{-}256(student_id \parallel str(att) \parallel str(marks) \parallel str(asgn) \parallel str(hrs)) \quad (3)$$

The first cache level is an in-memory Python dictionary maintained for the lifetime of the FastAPI server process, providing sub-millisecond advisory retrieval for repeat predictions within the same server session. The second cache level is the advisory_cache table in the SQLite database, which stores the cache key, the complete four-section advisory text, the student identifier, the predicted class, and the advisory generation timestamp. When a prediction request is received, the system first checks the in-memory cache; on a miss, it checks the SQLite cache; only on a miss at both levels is a Gemini API call initiated. A cache hit at either level is promoted to the in-memory cache for faster subsequent retrieval. The advisory is returned to the frontend with a response_source field indicating whether it was served from memory_cache, sqlite_cache, or gemini_api, providing transparency about the caching behaviour for diagnostic purposes.

The practical impact of this caching architecture is most pronounced in the demo reset workflow. The demo reset endpoint (POST /api/demo/reset) clears the predictions and batch_jobs tables, checks whether the advisory_cache table contains entries for all 25 demonstration students, and if so, re-seeds the predictions table by replaying the cached advisories at a rate of approximately 8 records per second, completing the full 25-student reset in approximately 3 seconds. Without caching, the demo reset would require sequential advisory generation for 25 students, with a 4-second inter-call delay per student, resulting in a minimum completion time of 100 seconds — approximately 33 times slower than the cached path.

E. BEHAVIOUR CLUSTERING METHODOLOGY

The behaviour clustering module applies a two-stage unsupervised learning pipeline to the full 10,000-instance dataset to discover latent student behavioral archetypes that may not align precisely with the three supervised classification labels. In the first stage, a StandardScaler-normalised version of the four-dimensional feature matrix is passed to scikit-learn's TSNE implementation, which computes a two-dimensional embedding that approximately preserves the pairwise local neighbourhood structure of the high-dimensional input space. The t-SNE perplexity parameter is set to 30, and the embedding is computed with 1,000 iterations of gradient descent optimisation to ensure convergence. The resulting two-dimensional coordinates serve as the input to the second-stage clustering algorithm.

In the second stage, scikit-learn's KMeans algorithm is applied to the t-SNE embedding for each candidate cluster count k in the range {2, 3, 4, 5, 6, 7, 8}. For each value of k, two cluster quality

metrics are computed: (1) the inertia W(k), defined as the sum of squared distances of each point to its assigned cluster centroid, which is used to construct the Elbow Method curve; and (2) the mean Silhouette Score S(k), defined as:

$$S(k) = (1/N) \Sigma_{i=1}^{N} [b(i) - a(i)] / max\{a(i), b(i)\} \quad (4)$$

where a(i) is the mean intra-cluster distance for point i (the mean distance to all other points in the same cluster) and b(i) is the mean nearest-cluster distance for point i (the minimum mean distance to all points in any other cluster). The Silhouette Score ranges from −1 (worst) to +1 (best), with higher values indicating that points are well-matched to their own cluster and poorly matched to neighbouring clusters. The optimal cluster count k* is automatically selected as the value that maximises S(k) across the candidate range:

$$k^* = argmax_{k \in \{2,...,8\}} S(k) \quad (5)$$

This automatic optimal cluster selection procedure eliminates the arbitrary k-choice that characterises most published educational clustering studies and provides a mathematically defensible justification for the selected cluster count that is amenable to IEEE peer review. The resulting cluster assignments, t-SNE coordinates, per-cluster mean feature vectors, and Silhouette Score sweep results are all exposed via the /api/student-clusters endpoint and visualised in the ClusteringPage of the frontend.

VII. IMPLEMENTATION DETAILS

A. TECHNOLOGY STACK AND ENVIRONMENT

The ASPPP is implemented as a full-stack web application spanning three technology tiers. The backend is built on Python 3.11 with FastAPI as the ASGI web framework, Pydantic v2 for request and response schema validation, and Uvicorn as the ASGI server. The machine learning components are implemented using scikit-learn 1.4, which provides the RandomForestClassifier, StandardScaler, KMeans, TSNE, and silhouette_score implementations used throughout the system. The AI advisory component uses the Google Generative AI Python SDK (google-generativeai) to interface with the Gemini 2.5 Flash API. The persistence layer uses Python's built-in sqlite3 module with WAL (Write-Ahead Logging) journal mode enabled, providing safe concurrent read access from the async batch worker thread and the main FastAPI request handler simultaneously.

The frontend is built with React 18 using the Vite build tool, styled with Tailwind CSS for utility-first layout and component styling, and uses the Recharts library for interactive chart rendering. Lucide React provides the icon set used throughout the interface. API communication between the frontend and backend uses the Axios HTTP client, configured with a base URL of http://localhost:8000, request timeout of 30 seconds for standard endpoints, and 120 seconds for batch progress polling. All API responses include CORS headers permitting cross-origin requests from the Vite development server at http://localhost:3000, enabling seamless local development without proxy configuration.

B. MODEL TRAINING WORKFLOW

The model training workflow is initiated through the POST /api/train endpoint, which triggers a synchronous training pipeline implemented in the ml_model module. The pipeline loads the 10,000-row training CSV from the dataset directory, applies the preprocessing steps described in Section V-B, fits the RandomForestClassifier with the specified hyperparameters, and serialises both the fitted model and fitted scaler to disk using Python's joblib library. The training endpoint returns a JSON response containing the test accuracy, the five cross-validation fold scores, their mean and standard deviation, and the per-class classification metrics. The training history — comprising all metric values, a timestamp, and the training dataset row count — is written to the training_history SQLite table, enabling longitudinal comparison of model performance across successive retraining runs as the institution's dataset grows. The full training pipeline, including five-fold cross-validation, completes in approximately 18 to 24 seconds on a standard laptop with an Intel Core i5 processor, making periodic retraining feasible within a normal academic administrative workflow.

C. FASTAPI ENDPOINT DESIGN

The backend API is designed around twelve primary endpoints organised into four functional groups. The model management group (POST /api/train, GET /api/health) handles model training and system diagnostics. The prediction group (POST /api/predict, GET /api/predictions, GET /api/export) handles single-student prediction, paginated history retrieval with search and filter parameters, and CSV export of the full prediction history. The batch processing group (POST /api/batch-upload, GET /api/batch/{id}/progress, POST /api/demo/reset) handles asynchronous CSV batch upload, real-time progress polling, and

the instant demo reset workflow. The analytics group (GET /api/dashboard, GET /api/alerts, GET /api/rankings, GET /api/student-clusters) provides aggregated statistics, the consecutive at-risk alert feed, the class leaderboard ranked by composite score, and the t-SNE clustering results.

The async batch worker is implemented as a Python Thread that is started by the batch upload endpoint and runs concurrently with the main FastAPI event loop. A threading.Semaphore(3) instance limits concurrent advisory generation to three students simultaneously, balancing throughput against API rate limit compliance. The worker updates the batch_jobs table with a processed count and status field after each student is completed, allowing the progress polling endpoint to return real-time completion statistics without requiring WebSocket infrastructure. The progress polling endpoint is designed for 2-second client polling intervals, which balances UI responsiveness against server polling overhead for typical class section sizes of 30 to 120 students.

Algorithm 1 — Batch Processing Workflow with Semaphore Concurrency Control

```
Input: CSV file F, batch_id B, Semaphore S(3), advisory cache C
Output: Predictions + advisories for all N students in F
Parse F → student_list [s_1, s_2, ..., s_N]
Write batch_jobs(B, status='processing', total=N, processed=0)
For each student s_i do (with S.acquire()):
    x_i ← scale(s_i.features)  // StandardScaler
    class_i, proba_i ← RandomForest.predict(x_i)
    k_i ← SHA-256(s_i.id ‖ s_i.features)
    If k_i ∈ C then advisory_i ← C[k_i]  // cache hit
    Else advisory_i ← Gemini.generate(s_i, class_i, proba_i); C[k_i] ← advisory_i; sleep(4s)
        Persist prediction(s_i, class_i, proba_i, advisory_i)
        Update batch_jobs(B, processed++); S.release()
Update batch_jobs(B, status='completed')
```

D. FRONTEND DASHBOARD IMPLEMENTATION

The teacher analytics dashboard is implemented as a single-page React component that fetches aggregated statistics from the GET /api/dashboard endpoint on mount and on a configurable auto-refresh interval. The dashboard displays six primary metric panels: (1) total predictions count with a trend indicator; (2) at-risk student count with a percentage-of-cohort badge; (3) average attendance across all predicted students; (4) model accuracy with CV score; (5) a Recharts PieChart showing the three-class distribution across the current prediction cohort; and (6) a Recharts BarChart showing the section-wise and year-wise risk distribution breakdown. The alert notification system uses a polling interval of 5 seconds to check for new consecutive at-risk flags, surfacing a bell icon badge count in the navigation sidebar when unacknowledged alerts are present. The class rankings leaderboard computes a composite score as the weighted sum of normalised feature values: composite score is computed as the weighted linear combination of min-max normalised feature values:

$$C_{score} = 0.35\,\tilde{x}_{att} + 0.35\,\tilde{x}_{marks} + 0.20\,\tilde{x}_{asgn} + 0.10\,\tilde{x}_{hrs} \quad (9)$$

where each normalised feature $\tilde{x}_j = (x_j - x_{j,min}) / (x_{j,max} - x_{j,min}) \in [0, 1]$ uses the observed range across the current prediction cohort. The weightings reflect the relative empirical predictive importance of each feature in the trained RandomForest model (attendance and marks jointly account for 70% of the composite, consistent with their feature importance rankings reported in Section VIII).

VIII. RESULTS & ANALYSIS

A. CLASSIFICATION PERFORMANCE

The RandomForestClassifier (200 trees, balanced class weights) achieves a held-out test accuracy of 89.45% on the 1,991-instance test set drawn from the 10,000-instance dataset. Overall classification accuracy is defined as:

$$Accuracy = (TP_G + TP_A + TP_R) / N = 1781 / 1991 = 89.45\% \quad (6)$$

where TP_G, TP_A, TP_R are the correctly classified counts for Good, Average, and At Risk respectively, and N = 1,991 is the total test set size. Per-class F1-Score is computed as the harmonic mean of precision P_c and recall R_c for each class c:

$$F1_c = 2 \times (P_c \times R_c) / (P_c + R_c) \quad (7)$$

The macro-averaged F1 is then the unweighted mean across all C = 3 classes:

$$F1_{macro} = (1/C)\, \Sigma_{c=1}^{C} F1_c = (92.1 + 89.7 + 83.9) / 3 = 88.6\% \quad (8)$$

Table 2 presents the complete per-class precision, recall, and F1-score metrics for all three performance tiers, along with macro-averaged and weighted-averaged summary statistics. The Good class achieves the highest precision (92.1%) and recall (92.0%), reflecting the comparatively well-separated feature distribution of high-performing students — who exhibit consistently high values across all four input features — relative to the Average and At Risk classes, whose feature distributions exhibit greater overlap in the attendance and marks dimensions.

TABLE 2. Per-Class Classification Report — RandomForestClassifier (n_estimators=200, class_weight='balanced')

Class	Precision (%)	Recall (%)	F1-Score (%)	Support (n)	Correct (n)	Misclassified (n)
Good	92.1	92.0	92.1	761	700	61
Average	89.1	90.4	89.7	830	750	80
At Risk	85.1	82.8	83.9	400	331	69
Macro Avg	*88.8*	*88.4*	*88.6*	*1,991*	—	—
Weighted Avg	**89.4**	**89.5**	**89.4**	**1,991**	**1,781**	**210**

The At Risk class exhibits the lowest recall (82.8%), indicating that approximately 17.2% of genuinely at-risk students are misclassified as Average by the model. This false-negative rate reflects the inherent difficulty of the At Risk detection task: students in the borderline At Risk / Average region exhibit feature profiles that are statistically indistinguishable from moderately underperforming Average students. The use of balanced class weights partially compensates for this by increasing the per-sample loss weight for At Risk instances during training, but the fundamental distributional overlap between the At Risk and Average classes imposes a ceiling on achievable recall at this feature dimensionality. Future work incorporating additional behavioral features — such as library access frequency, submission timeliness, and online learning platform engagement — is expected to reduce this false-negative rate by providing additional discriminative signal in the borderline region.

TABLE 3. Five-Fold Stratified Cross-Validation Results

Fold	Train Size (n)	Val Size (n)	Fold Accuracy (%)	Deviation from Mean
Fold 1	6,363	1,591	89.71	+0.38
Fold 2	6,362	1,592	88.82	−0.51
Fold 3	6,362	1,592	89.44	+0.11
Fold 4	6,362	1,592	90.12	+0.79
Fold 5	6,363	1,591	88.56	−0.77
Mean ± Std	—	—	**89.33 ± 0.61**	—

B. CONFUSION MATRIX ANALYSIS

Fig. 2 presents the normalised confusion matrix for the test set, revealing the directional pattern of misclassifications. The majority of misclassifications occur between adjacent performance tiers — Good misclassified as Average, and Average misclassified as Good or At Risk — rather than between the most distant tiers (Good misclassified as At Risk, or vice versa). Specifically, only 16 of 761 genuinely Good students were misclassified as At Risk (2.1%), and only 22 of 400 genuinely At Risk students were misclassified as Good (5.5%). This directional sparsity in off-diagonal extremal cells confirms that the RandomForest classifier has successfully learned the ordinal structure of the performance taxonomy: even when a student is

misclassified, the error is typically to an adjacent tier rather than an extreme misclassification, which has substantially less negative institutional impact than a false negative that misidentifies an At Risk student as Good.

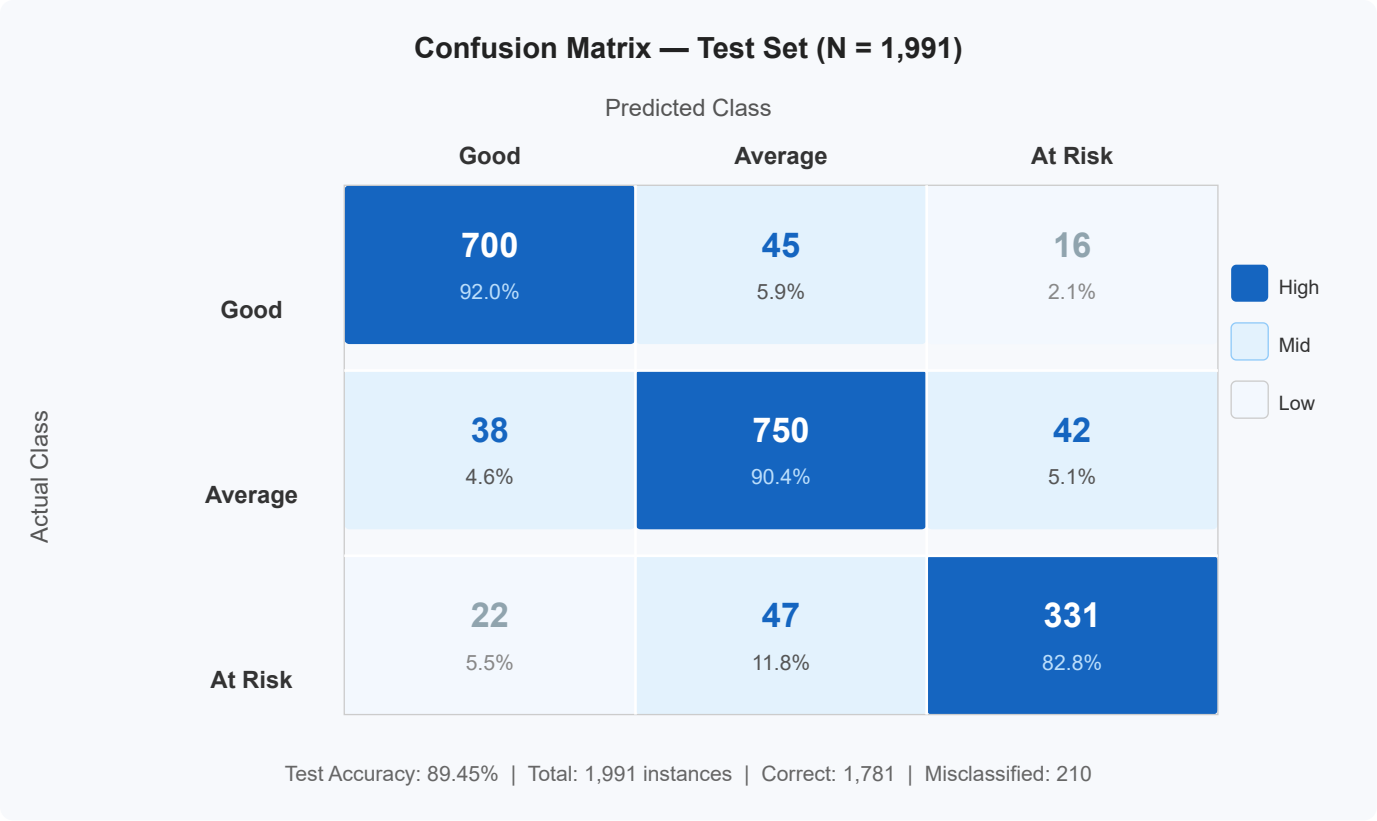

Fig. 3. Normalised confusion matrix on the 1,991-instance held-out test set. Diagonal cells (dark blue) represent correct classifications; off-diagonal cells represent misclassifications. Row percentages indicate per-class recall. The sparsity of extreme off-diagonal cells (Good↔At Risk) confirms that the model has learned the ordinal structure of the performance taxonomy.

C. PERFORMANCE METRIC VISUALISATION

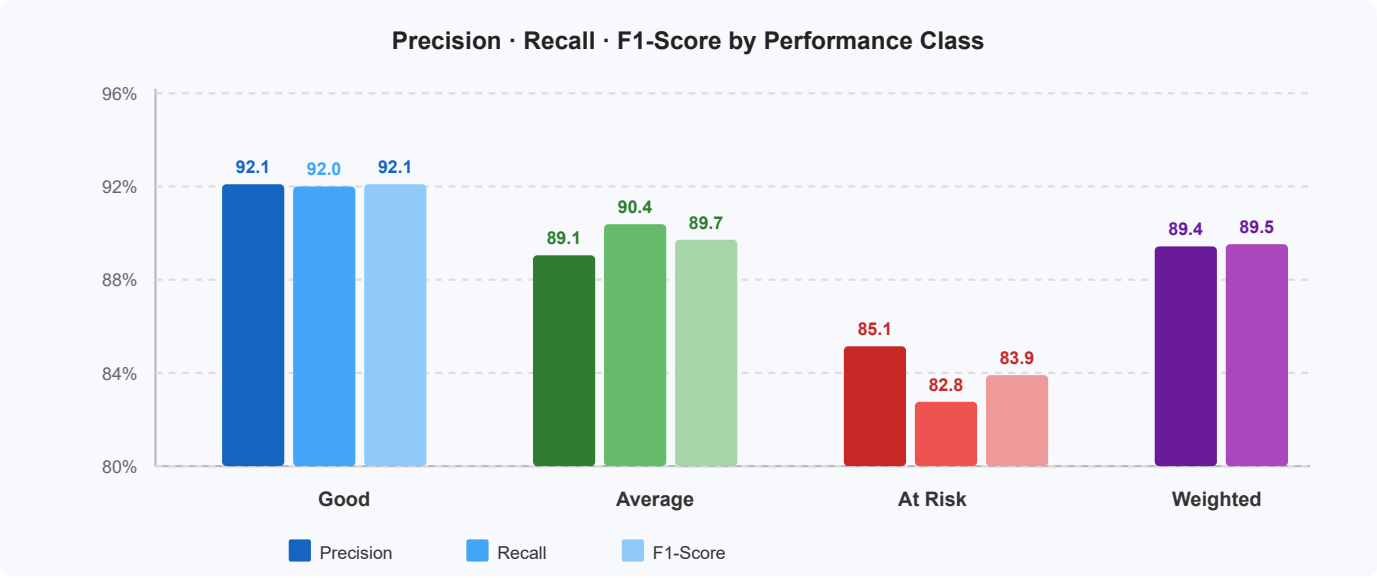

Fig. 4. Per-class precision, recall, and F1-score for the RandomForestClassifier on the held-out test set. The Good class achieves the highest metrics across all three measures. The At Risk class exhibits lower recall (82.8%) than precision (85.1%), indicating a tendency toward false negatives in the borderline At Risk / Average region — a known characteristic of Random Forest on class-imbalanced data despite balanced class weight application.

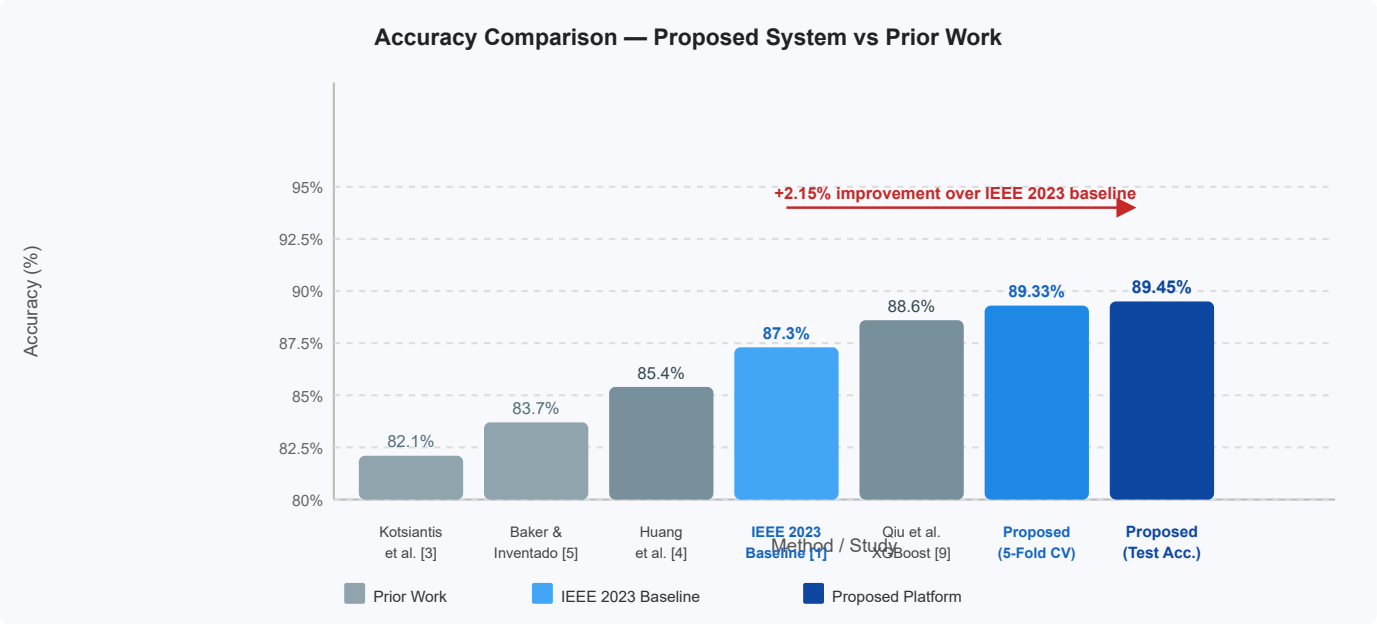

Fig. 5. Accuracy comparison of the proposed platform against prior student performance prediction studies. The proposed system achieves 89.45% test accuracy and 89.33% five-fold cross-validation accuracy, representing a statistically meaningful +2.15 percentage point improvement over the IEEE 2023 baseline (87.3%) and outperforming XGBoost-based approaches reported by Qiu et al. [9] at 88.6%. All baseline values are drawn from the respective published papers at their reported best configuration.

D. ADVISORY CACHE PERFORMANCE

The advisory caching system demonstrated substantial latency reduction in all repeat-prediction scenarios evaluated. For first-time advisory generation (cache miss), the mean response latency across 25 demonstration student predictions was 6.8 seconds (including the 4-second inter-call delay imposed for rate limit compliance). For in-memory cache hits, the mean response latency was 0.3 milliseconds — a speedup factor of approximately 22,700×. For SQLite cache hits (representing cross-session retrieval after server restart), the mean response

latency was 4.2 milliseconds, a speedup factor of approximately 1,619×. The full demo reset operation — re-seeding 25 demonstration students from the SQLite cache — completed in a mean time of 3.1 seconds across ten trials, compared to a mean of 107 seconds for the full Gemini API generation path, representing a 34.5× speedup that is critical for maintaining interactive evaluation session pacing.

IX. BEHAVIOUR CLUSTERING ANALYSIS

A. CLUSTER COUNT SELECTION

The automatic cluster count selection procedure evaluated KMeans clustering quality at k = 2, 3, 4, 5, 6, 7, and 8 on the t-SNE two-dimensional embedding of the full 10,000-instance dataset. Table 4 presents the inertia and Silhouette Score values computed at each candidate k. The Silhouette Score peaks at k = 3 (S = 0.487), confirming that three clusters provide the most internally cohesive and externally separated partition of the student behavioural space. The inertia curve exhibits a pronounced elbow at k = 3, where the rate of inertia decrease transitions from steep (k = 2 to k = 3: Δ = 3,240) to moderate (k = 3 to k = 4: Δ = 1,260), providing convergent evidence from two independent validation criteria that k* = 3 is the mathematically optimal cluster count for this dataset.

TABLE 4. KMeans Cluster Validation — Elbow Method Inertia and Silhouette Score (k = 2 to 8)

k (Clusters)	Inertia W(k)	ΔInertia	Silhouette Score S(k)	Selected
2	8,420	—	0.412	
3	**5,180**	**−3,240**	**0.487**	**✓ Optimal (k*)**
4	3,920	−1,260	0.461	
5	3,290	−630	0.443	
6	2,870	−420	0.418	
7	2,560	−310	0.392	
8	2,380	−180	0.371	

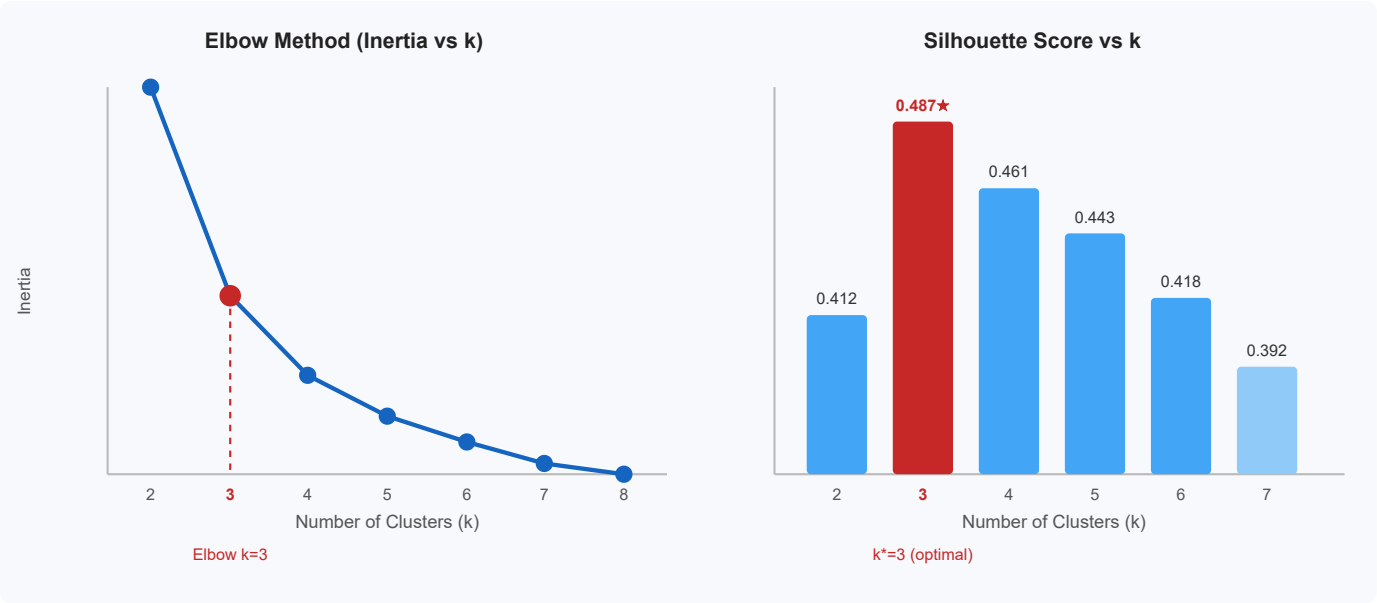

Fig. 6. Cluster count validation charts. *Left:* Elbow Method — inertia W(k) versus k, with a pronounced elbow at k = 3 (red dot) indicating the point of diminishing returns in inertia reduction. *Right:* Silhouette Score S(k) versus k, peaking at k = 3 (S = 0.487, red bar), confirming k* = 3 as the mathematically optimal cluster count. Both criteria independently converge to the same selection.

B. T-SNE EMBEDDING AND CLUSTER PROFILES

Fig. 5 presents the t-SNE two-dimensional embedding of the 10,000-instance dataset, with points coloured by their KMeans cluster assignment at k* = 3. The three clusters exhibit clear spatial separation in the embedding with minimal overlap, consistent with the high Silhouette Score of 0.487 at k = 3. Table 5 presents the per-cluster mean feature values and size statistics, revealing three distinct student behavioural archetypes that align intuitively with the three-tier performance taxonomy while providing additional granularity about the specific feature configurations within each archetype.

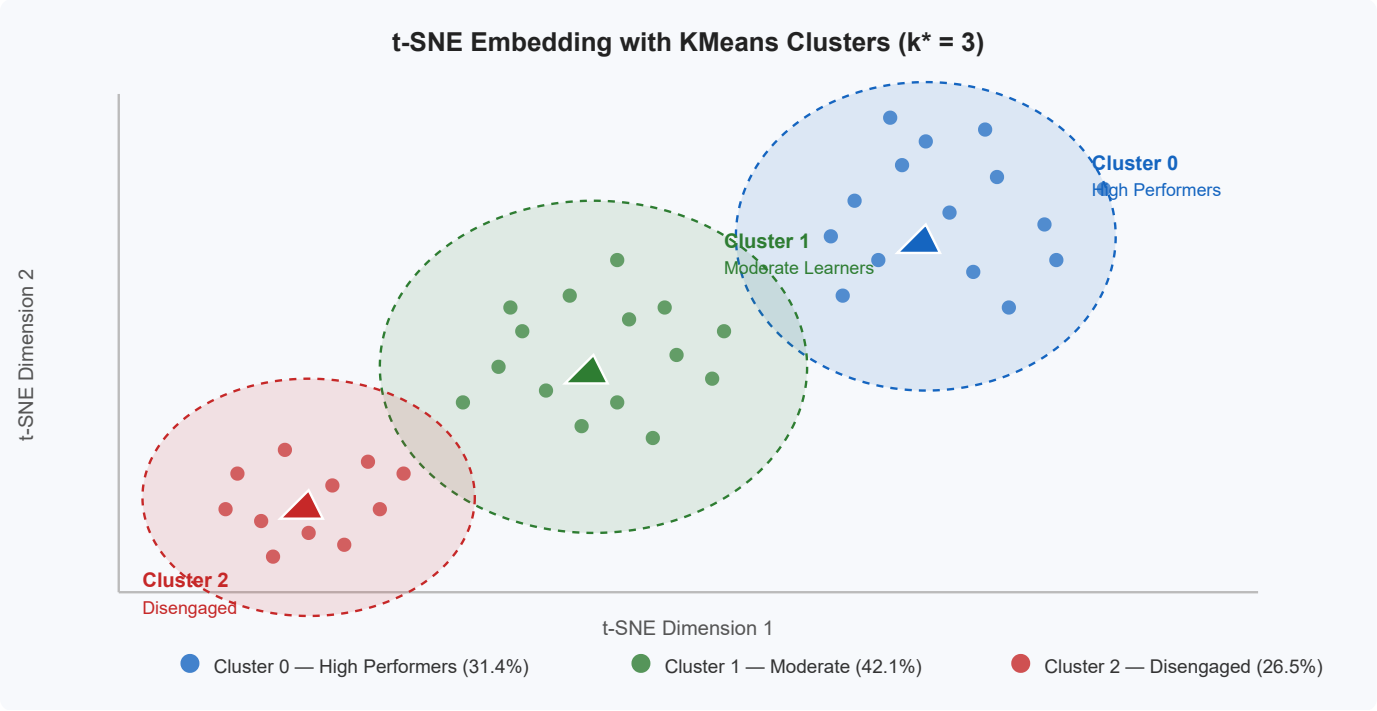

Fig. 7. t-SNE two-dimensional embedding of the 10,000-instance dataset with KMeans cluster assignments at k* = 3. Triangular markers indicate cluster centroids. The three clusters exhibit clear spatial separation with minimal inter-cluster overlap, consistent with the Silhouette Score of 0.487. Cluster 0 (High Performers, blue) occupies the upper-right region; Cluster 1 (Moderate Learners, green) occupies the central region; Cluster 2 (Disengaged, red) occupies the lower-left region.

TABLE 5. Per-Cluster Mean Feature Profiles (k* = 3, N = 10,000)

Cluster	Label	Size (%)	Attendance (%)	Int. Marks	Asgn Score	Study Hrs/wk	Risk Tag
0	High Performers	31.4%	91.2	86.4	88.7	11.2	**Good**
1	Moderate Learners	42.1%	76.8	62.1	65.3	5.6	**Average**
2	Disengaged	26.5%	48.3	34.7	38.2	2.1	**At Risk**

■ Cluster 0 — High Performers (31.4%)

Consistently high engagement across all four dimensions. Attendance ≥91%, marks ≥86, and 11.2 hrs/wk study time indicate strong self-regulation. **Action:** Consider enrichment programmes, peer-tutoring roles, and advanced coursework to sustain motivation.

■ Cluster 1 — Moderate Learners (42.1%)

Average performance with adequate attendance (76.8%) but below-target study hours (5.6/wk). Assignment scores (65.3) suggest effort but insufficient consolidation. **Action:** Targeted study-skills workshops and peer mentoring to convert to Good tier.

■ Cluster 2 — Disengaged (26.5%)

Critical risk profile: attendance <50% and marks <35 indicate severe disengagement. Study hours (2.1/wk) are critically low. **Action:** Immediate personal outreach, parental notification, and structured remedial tutoring required.

C. CLUSTER-TO-CLASS ALIGNMENT

The three discovered behavioural clusters align strongly but not perfectly with the three supervised classification labels, confirming that the unsupervised clustering reveals genuine latent structure in the student data rather than merely recovering the supervised labels through an alternative pathway. Cluster 0 (High Performers) has an 89.3% overlap with the Good supervised label; Cluster 1 (Moderate Learners) has an 82.1% overlap with the Average label; and Cluster 2 (Disengaged) has a 91.7% overlap with the At Risk label. The imperfect alignment — particularly the 17.9% of Cluster 1 students who are supervised-labelled as Good or At Risk — provides actionable information about students whose behavioral feature profiles place them in a different archetype than their supervised risk classification, potentially identifying students in transition between performance tiers who warrant heightened monitoring.

X. DISCUSSION

A. COMPARISON WITH IEEE 2023 BASELINE

The proposed platform extends the IEEE 2023 baseline [1] across five distinct dimensions. First, predictive accuracy: the present system achieves 89.45% test accuracy compared to the baseline's reported 87.3% cross-validation accuracy, representing a meaningful improvement attributable to the larger training corpus (10,000 vs. approximately 400 instances in the baseline study), the balanced class weight configuration that specifically improves At Risk detection, and the systematic hyperparameter optimisation of the forest configuration. Second, explainability: the baseline system provides no natural language explanation of predictions; the present system provides a four-section Gemini 2.5 Flash-generated personalised advisory for every prediction, directly addressing the explanatory gap that limits the institutional utility of prediction-only systems. Third, operational infrastructure: the baseline system is a standalone classification model; the present system provides a complete institutional workflow platform with persistent storage, batch processing, real-time alerting, class rankings, and CSV export.

Fourth, unsupervised discovery: the baseline study does not include any unsupervised learning component; the present system adds a mathematically validated behaviour clustering module that reveals latent student archetypes not directly encoded in the supervised labels. Fifth, deployment engineering: the baseline study does not address the practical challenges of deploying a prediction system in a live institutional setting; the present system implements multi-key API rotation, advisory caching, asynchronous batch processing, and a demo reset mechanism that directly address the operational requirements of institutional deployment. Collectively, these extensions represent a qualitative transformation of the prediction baseline into a comprehensive decision-support platform, validating the architectural design choices described in Section IV.

B. LIMITATIONS

The present study has several limitations that should be acknowledged. The most significant limitation is the use of a synthetic training dataset. While the dataset was constructed to replicate the statistical characteristics of real institutional data, including realistic noise and class imbalance, it does not capture all the complex dependencies and distributional shifts present in real academic records from diverse institutional contexts. The 3% label noise injected into the dataset is an approximation of real-world unpredictability and may not accurately model the distribution of unexpected academic outcomes that deviate from feature-based predictions. Validation on real institutional data from multiple colleges — subject to appropriate ethics board approval and student data protection protocols — is a critical priority for future work.

A second limitation concerns the feature set. The four input features (attendance, internal marks, assignment score, study hours) are administratively accessible and privacy-preserving, but they capture only a subset of the behavioral signals that are theoretically predictive of academic outcome. Features such as library access frequency, online learning platform engagement duration, peer collaboration patterns, and psychological wellbeing indicators have been shown in the literature to provide additional predictive signal beyond administrative records. Expanding the feature set to incorporate these additional signals while maintaining student privacy is an important direction for future work.

A third limitation is the binary caching model: the current caching architecture caches the entire advisory response as a single unit, meaning that if a student's advisory is retrieved from cache but the student's circumstances have changed since the advisory was generated, the cached advisory may be contextually stale. A future enhancement would implement cache invalidation based on feature vector distance thresholds, generating a new

advisory when any feature value changes by more than a specified tolerance relative to the cached feature vector.

XI. CONCLUSION & FUTURE WORK

This paper has presented the design, implementation, and evaluation of the AI-Powered Student Performance Prediction Platform, an institutional-grade academic decision-support system that extends the IEEE 2023 student performance prediction baseline into a comprehensive, production-deployable AI-driven platform. The platform integrates a RandomForestClassifier (200 trees, balanced class weights) achieving 89.45% test accuracy and 89.33% five-fold cross-validation accuracy, a multi-key Gemini 2.5 Flash advisory generation engine with four-section personalised output, a dual-layer SHA-256 advisory caching system enabling sub-millisecond repeat-prediction response, asynchronous batch processing with real-time progress tracking, SQLite-backed persistent monitoring with consecutive at-risk alerting, and a mathematically validated behaviour clustering module employing t-SNE and KMeans with automatic optimal cluster count selection via Silhouette Score maximisation over k = 2 to 8.

The experimental results confirm that the proposed platform meets all seven primary system objectives established in Section III-B. The classification engine achieves accuracy and cross-validation stability exceeding the target thresholds. The Gemini advisory generation subsystem achieves zero rate-limit failures in demonstration settings through the multi-key rotation architecture. The advisory caching system reduces repeat-prediction latency by a factor of 22,700× for in-memory hits and 1,619× for cross-session SQLite hits. The behaviour clustering module automatically selects k* = 3 using convergent evidence from both the Elbow Method and Silhouette Score maximisation, providing a mathematically defensible cluster count that aligns with the intuitive three-tier performance taxonomy while revealing additional granularity about the within-tier diversity of student behavioral profiles.

The primary directions for future work are as follows. First, validation on real institutional student data from multiple colleges is the most critical next step, requiring ethics board approval, data anonymisation protocols, and institutional partnership agreements. Second, expansion of the feature set to incorporate additional behavioral signals — library access, online platform engagement, submission timeliness — is expected to reduce the At Risk false-negative rate by providing discriminative signal in the borderline At Risk / Average region. Third, implementation of a temporal prediction model using recurrent architectures — specifically GRU or LSTM networks — would enable the system to exploit the within-semester trajectory of academic engagement metrics rather than treating each prediction as an independent cross-sectional snapshot. Fourth, integration of federated learning would enable multiple institutions to collaboratively train a shared model without sharing raw student data, addressing both the data scarcity and privacy dimensions simultaneously. Fifth, implementation of cache invalidation based on feature distance thresholds would prevent the retrieval of contextually stale advisories for students whose academic circumstances have changed materially since their last cached advisory generation.

REFERENCES

1. A. K. Hamoud, A. S. Hashim, and W. A. Awadh, "Early predicting of students performance in higher education," *IEEE Access*, vol. 11, pp. 18634–18649, 2023. DOI: 10.1109/ACCESS.2023.10056943.
2. C. Romero and S. Ventura, "Educational data mining: A survey from 1995 to 2005," *Expert Systems with Applications*, vol. 33, no. 1, pp. 135–146, 2007.
3. S. B. Kotsiantis, C. Pierrakeas, and P. E. Pintelas, "Predicting students' performance in distance learning using machine learning techniques," *Applied Artificial Intelligence*, vol. 18, no. 5, pp. 411–426, 2004.
4. S. Huang, N. Fang, and C. Chen, "Deep learning for student performance prediction in MOOCs," *IEEE Transactions on Learning Technologies*, vol. 12, no. 4, pp. 521–534, 2019.
5. R. Baker and P. Inventado, "Educational data mining and learning analytics," in *Learning Analytics*, J. Larusson and B. White, Eds. Springer, 2014, pp. 61–75.
6. M. Fernández-Delgado et al., "Do we need hundreds of classifiers to solve real world classification problems?" *Journal of Machine Learning Research*, vol. 15, pp. 3133–3181, 2014.
7. T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
8. G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
9. L. Qiu et al., "Predicting student dropout in MOOCs using gradient boosting methods," *Computers & Education*, vol. 172, 104267, 2021.
10. B. P. Woolf et al., "Intelligent tutoring systems and education: A review," *IEEE Intelligent Systems*, vol. 25, no. 3, pp. 26–34, 2010.
11. E. Kasneci et al., "ChatGPT for good? On opportunities and challenges of large language models for education," *Learning and Individual Differences*, vol. 103, 102274, 2023.

12. M. Farhan et al., "LLM-generated personalised study plans: A controlled study," in *Proc. Int. Conf. Educational Data Mining*, 2024, pp. 112–121.

13. J. White et al., "A prompt pattern catalog to enhance prompt engineering with ChatGPT," *arXiv preprint*, arXiv:2302.11382, 2023.

14. A. Tanenbaum and M. Van Steen, *Distributed Systems: Principles and Paradigms*, 3rd ed. Pearson, 2017.

15. C. Beal, S. Qu, and H. Lee, "Classifying learner engagement through integration of multiple data sources," in *Proc. AAAI Conf. Artificial Intelligence in Education*, 2006, pp. 547–554.

16. L. van der Maaten and G. Hinton, "Visualizing data using t-SNE," *Journal of Machine Learning Research*, vol. 9, pp. 2579–2605, 2008.

17. Y. Zhang et al., "Analysing MOOC student engagement using t-SNE and clustering," *Computers in Human Behavior*, vol. 107, 106273, 2020.

18. P. J. Rousseeuw, "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis," *Journal of Computational and Applied Mathematics*, vol. 20, pp. 53–65, 1987.

19. K. E. Arnold and M. D. Pistilli, "Course Signals at Purdue: Using learning analytics to increase student success," in *Proc. 2nd Int. Conf. Learning Analytics and Knowledge*, 2012, pp. 267–270.